Project Assignment — Software Developer

ATM Cash Management Backend

Your task is to build the cash management backend for a bank's ATM network. Only the backend is in scope; no user interface is required.

The assignment has two parts. **Part A** describes the core business requirements. **Part B** adds the kind of requirements you would meet in a real banking system. Each item in Part B is assessed on its own. If you cannot finish everything, a smaller set of items that are complete and working is worth more than every item half done.

You may use AI tools freely.

Part A — Core business requirements

1. The supported currencies are TRY, USD and EUR.
2. Every currency has only 20, 50, 100 and 500 banknotes. Coins are not supported.
3. Each ATM has 5 cassettes. A cassette holds at most 100 banknotes.
4. One cassette is a **deposit-only cassette**. It accepts any denomination in any currency, but cash is never dispensed from it.
5. The other 4 cassettes are **recycling cassettes**. Each one accepts and dispenses a single denomination of a single currency (for example, TRY 100).
6. At least **one** and at most **two** of the recycling cassettes may be configured for a foreign currency (USD or EUR).
7. Before an ATM can go into service, its cassettes must be configured and the cash loaded into them by hand must also be recorded in the system.
8. At regular intervals the cash in the cassettes is counted by hand and reconciled against the system. Any difference between the system figures and the physical count must be recorded in the system.
9. When a customer requests a withdrawal, the system returns how many notes of each denomination will be dispensed. If the requested amount cannot be paid exactly, the system returns the closest amount it can pay, together with its note breakdown.
10. The withdrawal is carried out once the customer confirms it.
11. For a deposit, the system receives how many notes of each denomination were inserted. Notes that have a matching recycling cassette go to that cassette; all other notes go to the deposit cassette.
12. An operations officer must be able to see, at any time, the number of notes of each denomination in an ATM and its total balance.

Part B — Additional requirements

B1. Multiple ATMs

The system manages a network of ATMs. Every rule in Part A applies per ATM.

B2. ATM lifecycle

An ATM is always in one of the following states:

`UNCONFIGURED → CONFIGURED → LOADED → IN_SERVICE ⇄ MAINTENANCE`

- Cassette configuration, cash loading and reconciliation are allowed **only in MAINTENANCE** (or in UNCONFIGURED/CONFIGURED during initial setup).
- Withdrawals and deposits are allowed **only in IN_SERVICE**.
- An invalid state transition, or an operation attempted in the wrong state, must be rejected with a meaningful error.
- The same currency and denomination may be assigned to two recycling cassettes. In that case, notes are dispensed from **the cassette with the higher fill ratio** first.

B3. Optimal note selection

- For a withdrawal, the system must find the breakdown that pays the requested amount with **the fewest notes**, given the limited number of notes in each cassette.
- If the amount cannot be paid exactly, the system returns two alternatives:
  - **Lower alternative:** the largest payable amount below the requested amount.
  - **Upper alternative:** the smallest payable amount above the requested amount. It is offered only if it does not exceed the customer's daily limit.
- The following acceptance tests must be included in your project **as automated tests** and must pass:

#	Currency	Notes available	Requested	Expected result
1	TRY	100×0, 50×3, 20×10	160	Exact: 50×2 + 20×3
2	TRY	50×1, 20×2	80	No exact match. Lower: 70 (50×1 + 20×1), Upper: 90 (50×1 + 20×2)
3	TRY	100×5, 20×10	130	No exact match. Lower: 120 (100×1 + 20×1), Upper: 140 (100×1 + 20×2)
4	USD	100×2	250	No exact match. Lower: 200 (100×2), Upper: none
5	TRY	500×1, 100×0, 20×5	600	Exact: 500×1 + 20×5
6	TRY	100×1, 50×4, 20×5	200	Exact: 100×1 + 50×2 (not 4×50, and not 100 + 5×20)
7	TRY	500×1, 100×3, 20×20	1000	Exact: 500×1 + 100×3 + 20×10

#	Currency	Notes available	Requested	Expected result
8	TRY	100×3, 50×1, 20×1	110	No exact match. Lower: 100 (100×1), Upper: 120 (100×1 + 20×1)

> Note: A simple approach that starts with the largest denomination and works downwards gives the wrong answer for some of these tests.

B4. Two-step withdrawal: quote and confirm

1. **Quote:** The customer requests an amount. The system returns the exact breakdown or the alternatives, and **reserves** the notes it has offered.
2. **Confirm:** The customer either confirms or cancels the quote.

Rules: - A quote is valid for **60 seconds**. When a quote expires, its reservation is released automatically. - Reserved notes cannot be offered in any other quote. - The confirm request carries an `Idempotency-Key` header. A repeated request with the same key must return the same result, and **the cash must not be debited twice**. - An expired or cancelled quote cannot be confirmed.

B5. Daily limit per card

- The daily withdrawal limit per card is **TRY 20,000, USD 1,000 and EUR 1,000**, tracked separately for each currency.
- The limit is checked at the quote step. Amounts that are reserved but not yet confirmed count towards the limit.
- Card numbers must not be stored in plain text.

B6. Concurrency

Two quote requests arriving at the same ATM at the same moment must never reserve the same physical note twice. **Show with a test** that your solution handles this, for example an integration test that sends parallel requests.

B7. Deposit details

- In addition to the list of notes, a deposit request includes the number of notes the machine could not recognise. Unrecognised notes are returned to the customer and are not part of the transaction.
- If a recycling cassette becomes full, the overflow goes to the deposit cassette.
- If the deposit cassette is also full, the notes that do not fit are returned to the customer.
- The response shows, for each note, which cassette it went to, and lists separately any notes that were returned.

B8. Cassette movement ledger

- Every change to a cassette is stored as an **immutable movement record**. Movement types: LOAD, UNLOAD, WITHDRAWAL, DEPOSIT, RESERVATION, RESERVATION_RELEASE, RECONCILIATION_ADJUSTMENT.

- A cassette's current balance must be reproducible from its movements.
- Provide a **consistency check** endpoint that compares the stored balances with the balances calculated from the movements and reports any differences.

B9. Reconciliation with four-eyes approval

- During reconciliation, the counted number of notes is entered for each cassette. The system calculates the difference and creates a RECONCILIATION_ADJUSTMENT movement.
- If the total TRY value of the differences **exceeds TRY 1,000**, the reconciliation moves to an "awaiting approval" state. The ATM cannot return to service until **a different user** approves it. The user can simply be identified by a request header (`X-User`); authentication is not required.
- The system keeps an exchange-rate table for the TRY value calculation. You may provide the initial rates as seed data.

B10. Reporting and alerts

- **Operations report:** per ATM and per cassette, the note count, the reserved count and the available count, along with totals per currency and the overall total in TRY.
- **Low-cash alert:** when the available count in a recycling cassette falls below 10% of its capacity, an alert event is created. The event is written to an outbox table **within the same database transaction** as the operation that triggered it. A background service (hosted service) reads events from the outbox, logs them and marks them as sent.

Technical requirements

- ASP.NET Core REST Web API
- EF Core code-first, with migrations
- A relational database (SQL Server or PostgreSQL)
- Dependency injection
- A containerised application that starts with `docker compose up` (API and database). Migrations must be applied automatically on startup.
- Swagger/OpenAPI documentation
- A consistent error model (ProblemDetails) with meaningful error codes
- A health check endpoint
- **Automated tests:**
  - Unit tests for the acceptance tests in B3
  - At least one integration test that runs against a real database
- **Postman collection** that can run the end-to-end scenario in order: create ATM → configure cassettes → load cash → put in service → request quote → confirm → deposit → switch to maintenance → reconcile → report

Deliverables

1. A Git repository (link or zip)
2. A **README.md** in the repository root with the following sections:
   - How to run the project and its tests

    - The architecture and the key design decisions, with your reasoning. Examples: the note selection algorithm, the concurrency approach, idempotency, the movement ledger.
    - **Assumptions:** the points you found ambiguous in the requirements and the decisions you made about them
    - Unfinished items and known limitations
    - If you used AI tools, a short note: which tools you used, what for, and what you corrected in their output
3. The Postman collection (and environment file, if any)